

Redrob Candidate Ranker

India Runs: Data & AI Challenge

Team PJ2001-IND

1. The Problem: Keywords vs. True Fit

Recruiters spend countless hours parsing through 100,000+ candidates but still miss the right person because legacy hiring tools rely on **rigid keyword filtering**.

- **The Trap:** A "Marketing Manager" with "AI" sprinkled in their summary will rank higher than a seasoned engineer who uses alternative terminology.
- **The Reality:** True fit comes from understanding career trajectories, behavioral signals, and semantic relevance to the role.

2. Our Solution: A 4-Stage Intelligent Pipeline

We built a CPU-optimized, scalable pipeline that ranks 100,000 candidates in under **18 seconds** without needing external API calls.

1. Hard Filtering & Honeypot Evasion
2. Behavioral Scoring (Redrob Signals)
3. BM25 Keyword Matching
4. Semantic Re-Ranking (Local NLP)

3. Stage 1: Hard Filtering & Honey pots

To dramatically cut down the search space and avoid traps:

- **Honey pot Evasion:** We built rules to instantly disqualify candidates who claim expert proficiency in skills they have 0 months of experience using.
- **Title Validation:** Filtered out keyword-stuffing non-engineers (e.g., Marketing Managers).
- **Service Company Check:** Aligned with the Job Description's preference for product-company experience over pure service-company backgrounds.

4. Stage 2: Behavioral & BM25 Scoring

- **Redrob Behavioral Signals:** A candidate might look perfect on paper, but if they haven't logged in for 6 months or respond to 5% of messages, they are effectively unavailable. We multiply scores based on `recruiter_response_rate`, `github_activity_score`, and `notice_period_days`.
- **BM25 Search:** A blazing-fast probabilistic information retrieval algorithm that scores resumes against core JD requirements (e.g., `embeddings`, `retrieval`, `vector database`).

5. Stage 3: Semantic Re-Ranking

For the top 1000 candidates filtered by BM25 and Behavioral metrics, we apply true AI understanding:

- **Local NLP Model:** We deployed `sentence-transformers/all-MiniLM-L6-v2` locally to generate dense embeddings.
- **Cosine Similarity:** Computed the semantic distance between the candidate's career summary and the core requirements of the JD.
- **Result:** We capture candidates who describe their work conceptually (e.g., "built a recommendation engine") even if they don't use the exact keywords requested.

6. Stage 4: Tie-Breaking & Factual Reasoning

- **Strict Tie-Breaking:** Deterministically handled tied scores by sorting via `candidate_id` as per challenge guidelines.
- **Dynamic Reasoning Generator:** To assist human reviewers in Stage 4, we generate a 1-2 sentence factual explanation for the top 100 candidates based entirely on their real profile data (e.g., Years of Experience, Notice Period, Response Rate). No hallucinations.

7. Final Results & Impact

- **Scale:** Successfully evaluated **100,000 candidates**.
- **Speed:** Total runtime of **17.62 seconds** on a standard CPU.
- **Efficiency:** Fits comfortably within the 16GB RAM / 5-minute constraint.
- **Output:** A pristine `team_PJ2001_IND.csv` containing the Top 100 highest-quality, verifiable fits for the Senior AI Engineer role.

Building what next India runs on.